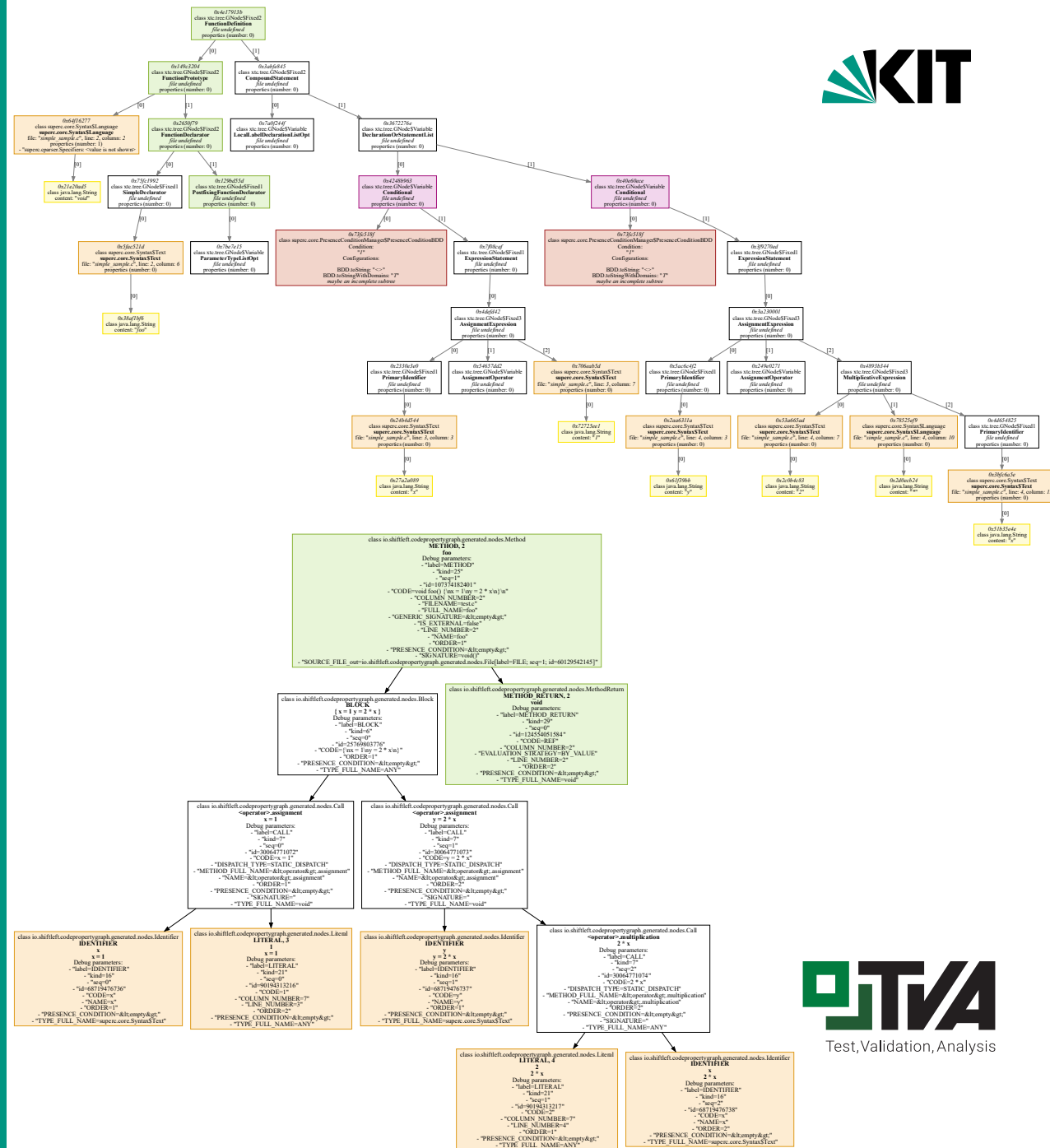


Family-Based Vulnerability Discovery for Highly-Configurable Software Systems

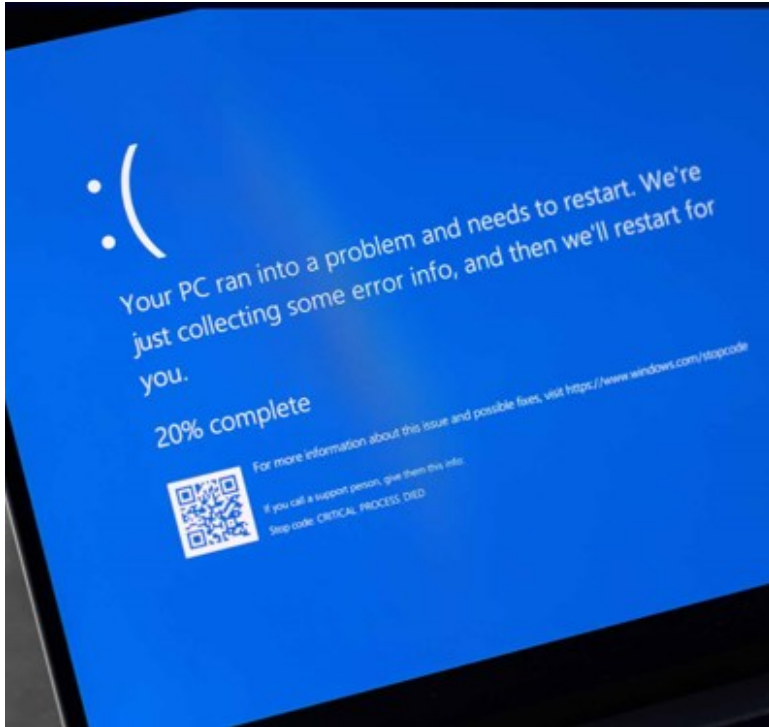
Supervisor: Tim Bächle

Chaoyang Wei, Fynn Tschacher, TVA KASTEL,
23 September 2026



1. Motivation
2. Approach
3. Implementation
4. Current State

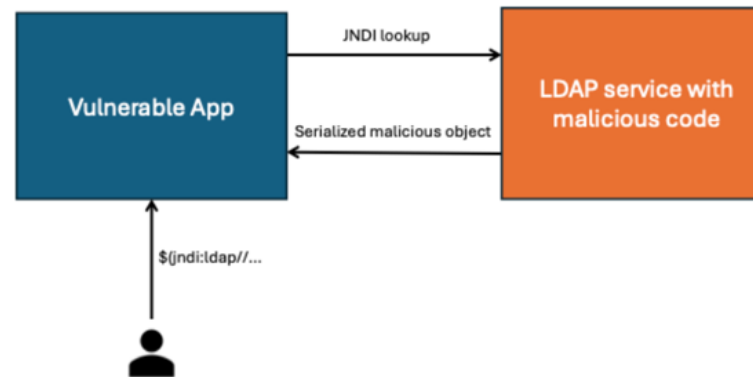
1.1 Motivation



CrowdStrike (2024) [Be+24]

- CrowdStrike Falcon update
- global Windows BSOD

Log4j exploitation flow diagram



Log4Shell (2021) [Tz+24]

- widely used library (log4j)
- RCE via JNDI lookup



Heartbleed (2014) [Go+14]

- optional OpenSSL feature (compile-time)
- memory disclosure

1.2 Problem Description

Classical (Q-)SAST assumption

- **(Q-)SAST:** analyze code without executing it
- One configuration $\Rightarrow$ one variant
- One variant/configuration, code without variability

Configurable Sample Code:

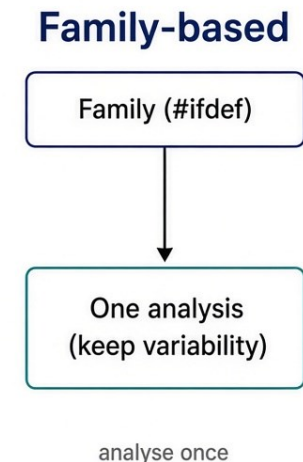
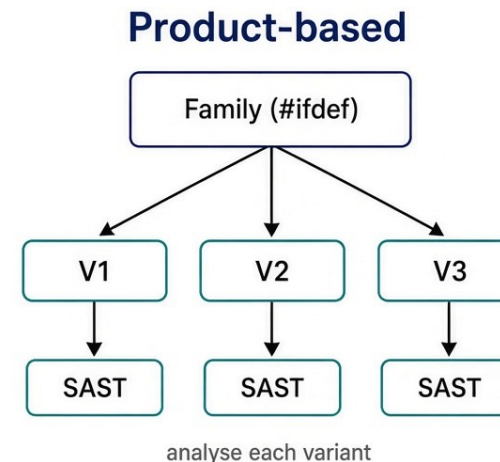
```
1. void foo() {  
2.     int x = source(); // Attacker-controlled.  
3.     if(x < MAX){ // Does not enforce x >= 0.  
4.         int y = 0;  
5.         #ifdef CONFIG_PROCESS_INPUT  
6.             y = 2 * x;  
7.             #ifdef CONFIG_SEND_DATA  
8.                 sink(y); // Security-sensitive operation.  
9.             #endif  
10.        #endif  
11.        // ...  
12.    }  
13. }
```

Problem

Vulnerability only if `CONFIG_PROCESS_INPUT` $\wedge$ `CONFIG_SEND_DATA`
 $\rightarrow$ **invisible in most single variants**

Highly-Configurable Software Systems

- **Definition:** one code base with configurable features
 - Retaliated with `#ifdef`-instructions
- **Why:** many variants shared code a lot of code



2.1 Q-SAST Lifting

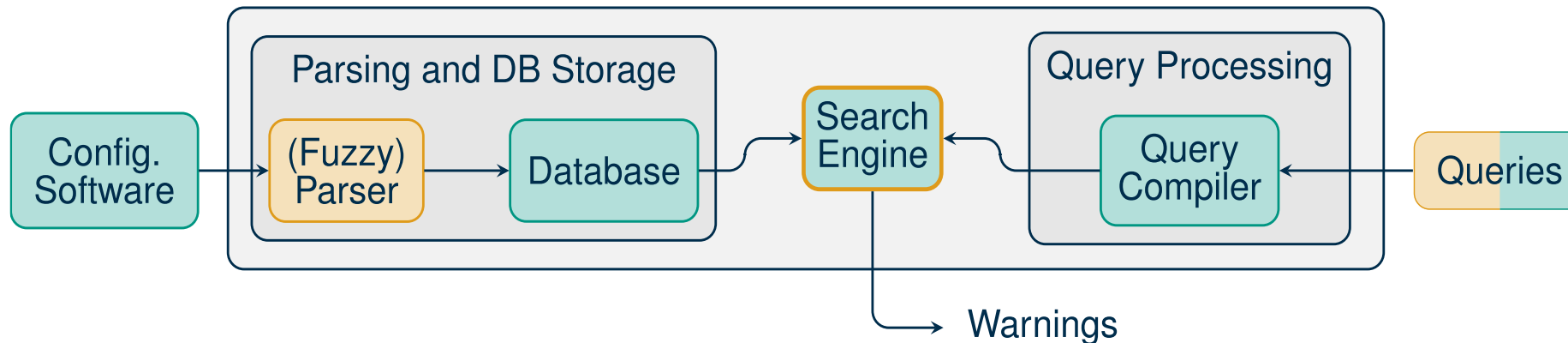
Product-based Analyze

- Analyze each software configuration individually
- Does not scale with number of configuration parameters

(Q-)SAST Lifting By Extension

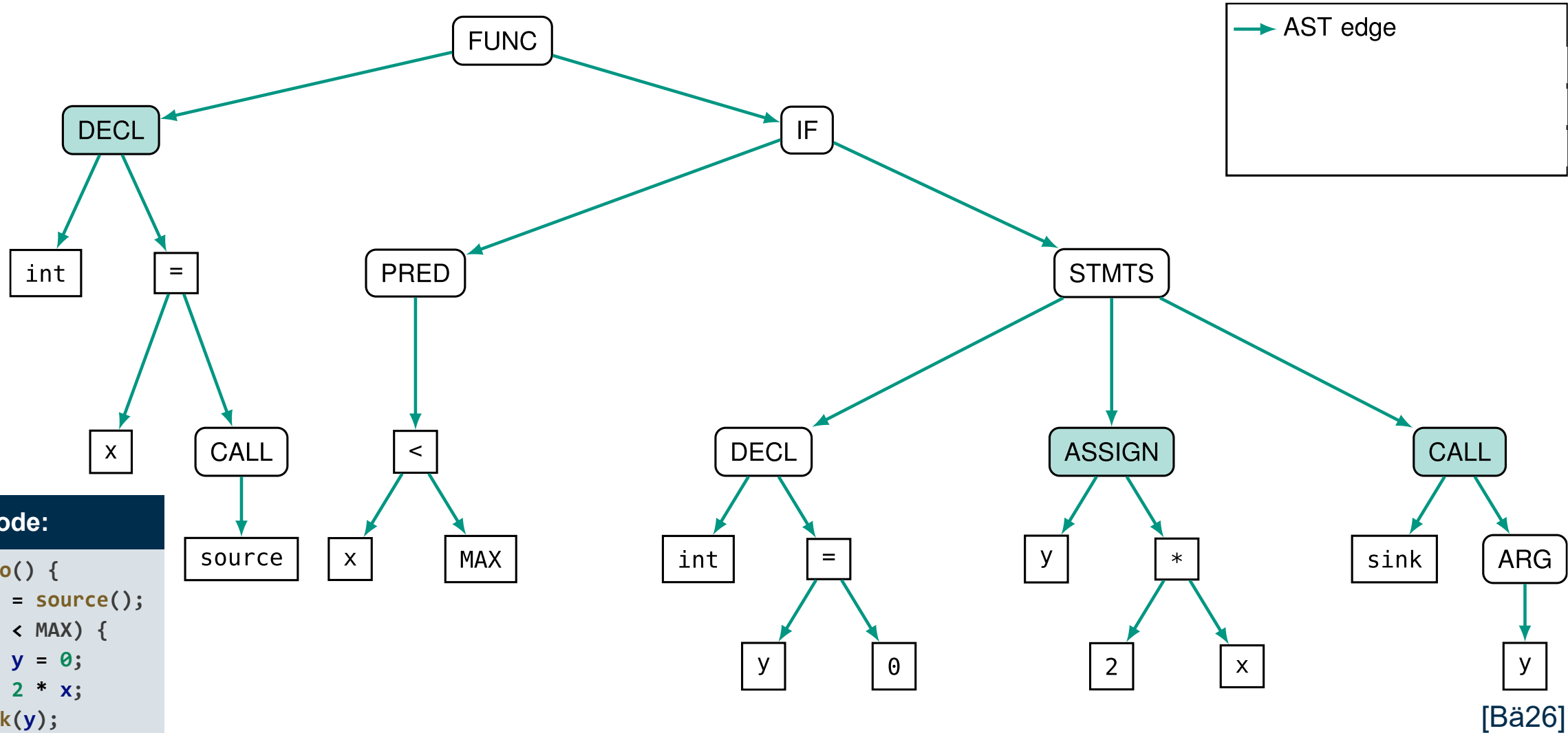
- Analyze the complete family at once
- Make the Q-SAST representation variability-aware
- Extend the parser, data representation and query compiler

Typical structure of Q-SAST tools as described by Li et al. [Li+24]

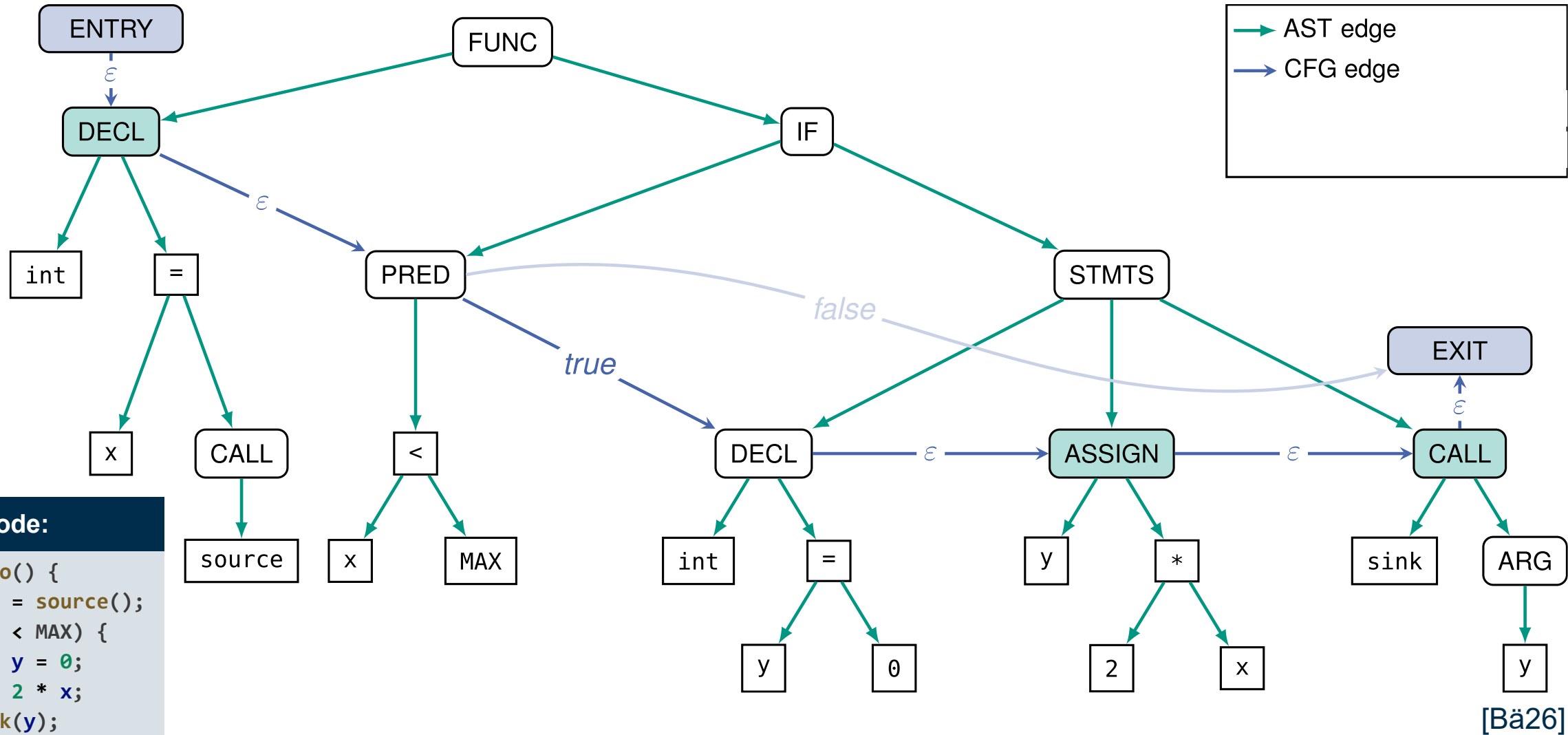


- Alterations required
- Alterations may be required
- Alterations optional
- Does not require alterations

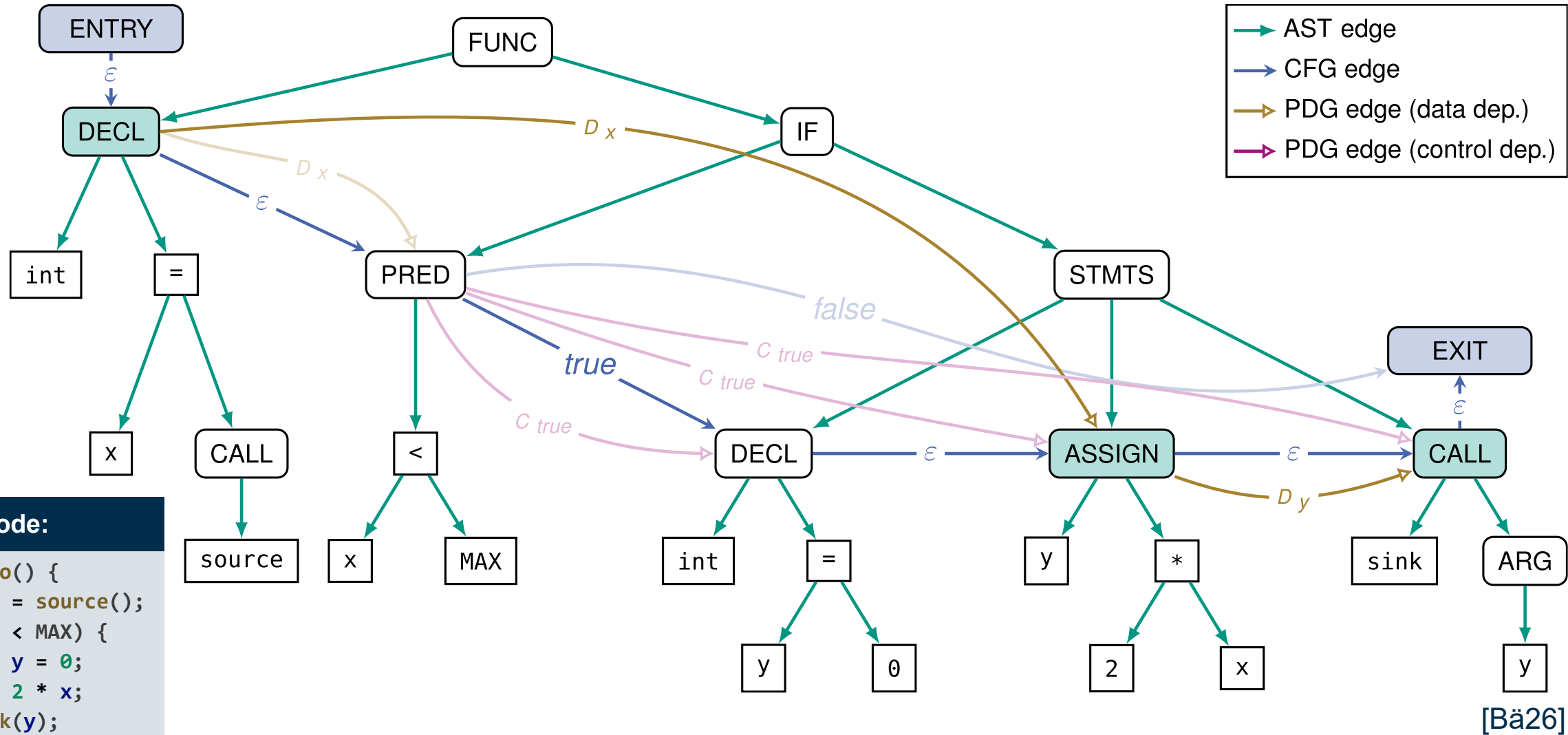
2.2 Data Structure



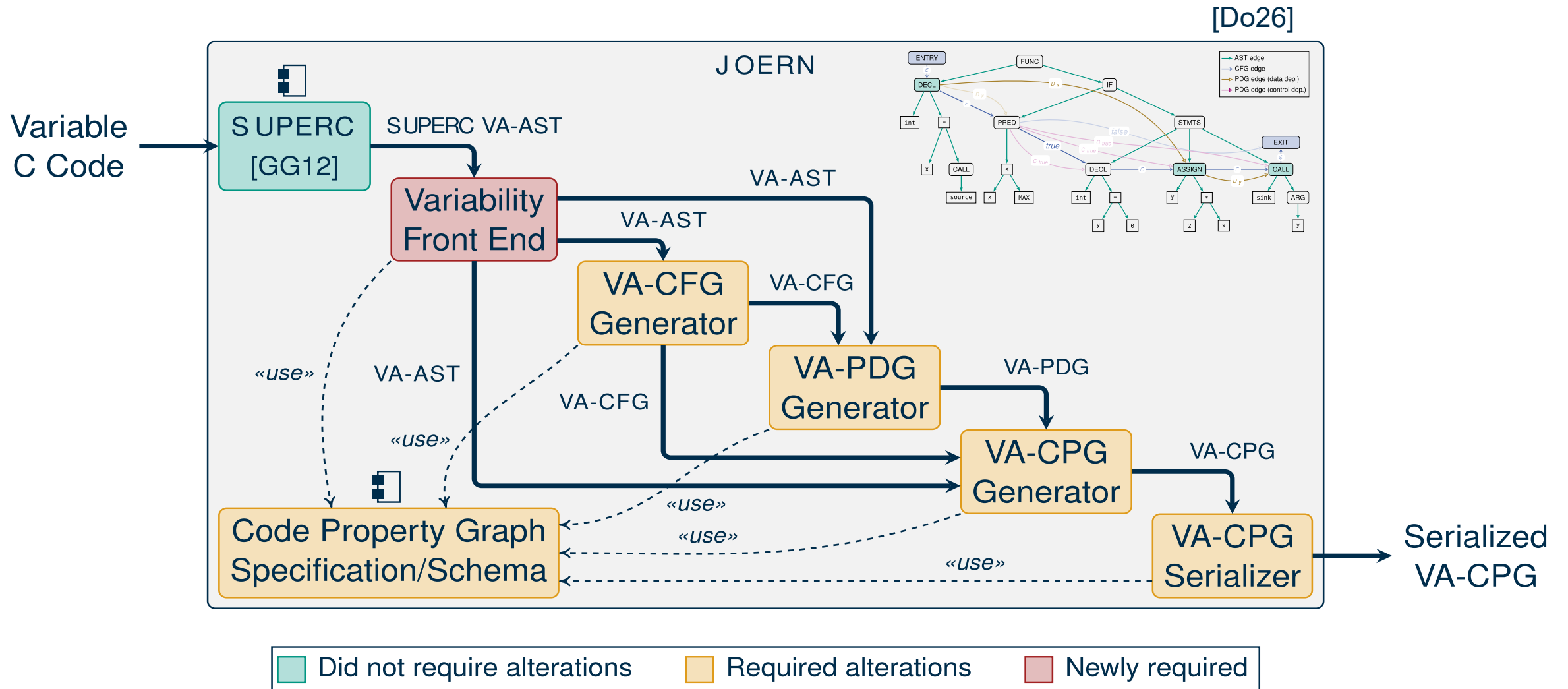
2.2 Data Structure



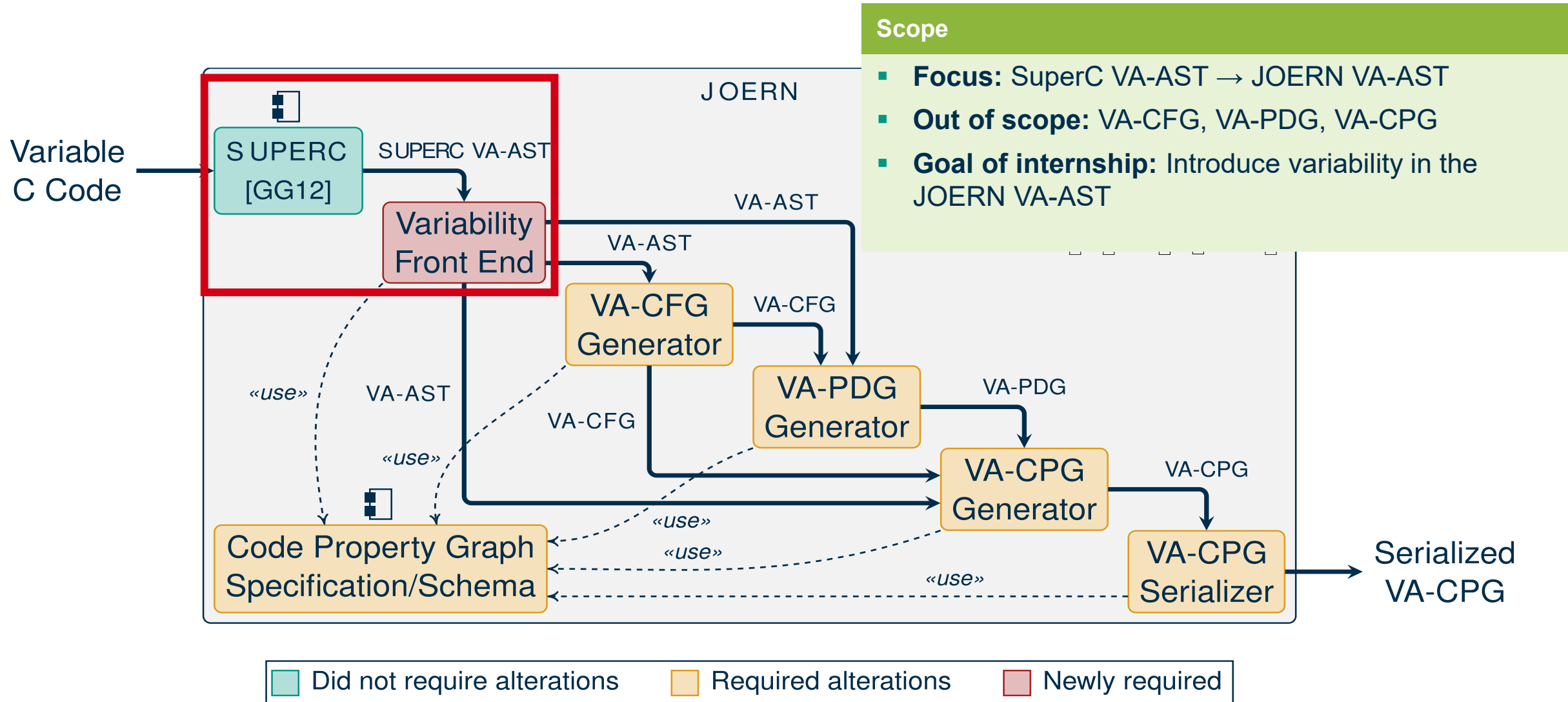
2.2 Data Structure



2.3 Processing Pipeline



2.3 Processing Pipeline



3.1 Translation C-Features

1. Unary operators

Operators like ++ or !

2. Binary operators

Operators like + or &&

3. Variable declaration

4. Multi-variable declaration

5. typedef and struct

typedef and struct declarations and initialization

6. enum

enum declarations

7. Simple parameterized macros

definition of macros

8. Function call

9. Function declaration

10. Multi-function handling

11. Multi-file support

Create one VA-AST for multiple C files

12. Code generation

Code reconstruction from SuperC VA-AST

13. Code block handling

sequence of instruction
e.g. if-block

14. Conditional macros

Preprocessor configurations options e.g.:
#if, #elif, #else,
#endif, ...

15. if statements

16. switch-case statements

17. while and do-while loop

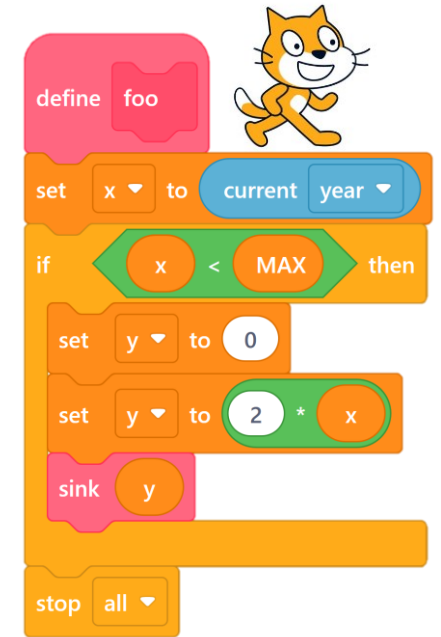
18. for loop

19. break and continue

20. return statements

21. goto and goto-labels

22. Assembler instructions



Discovery

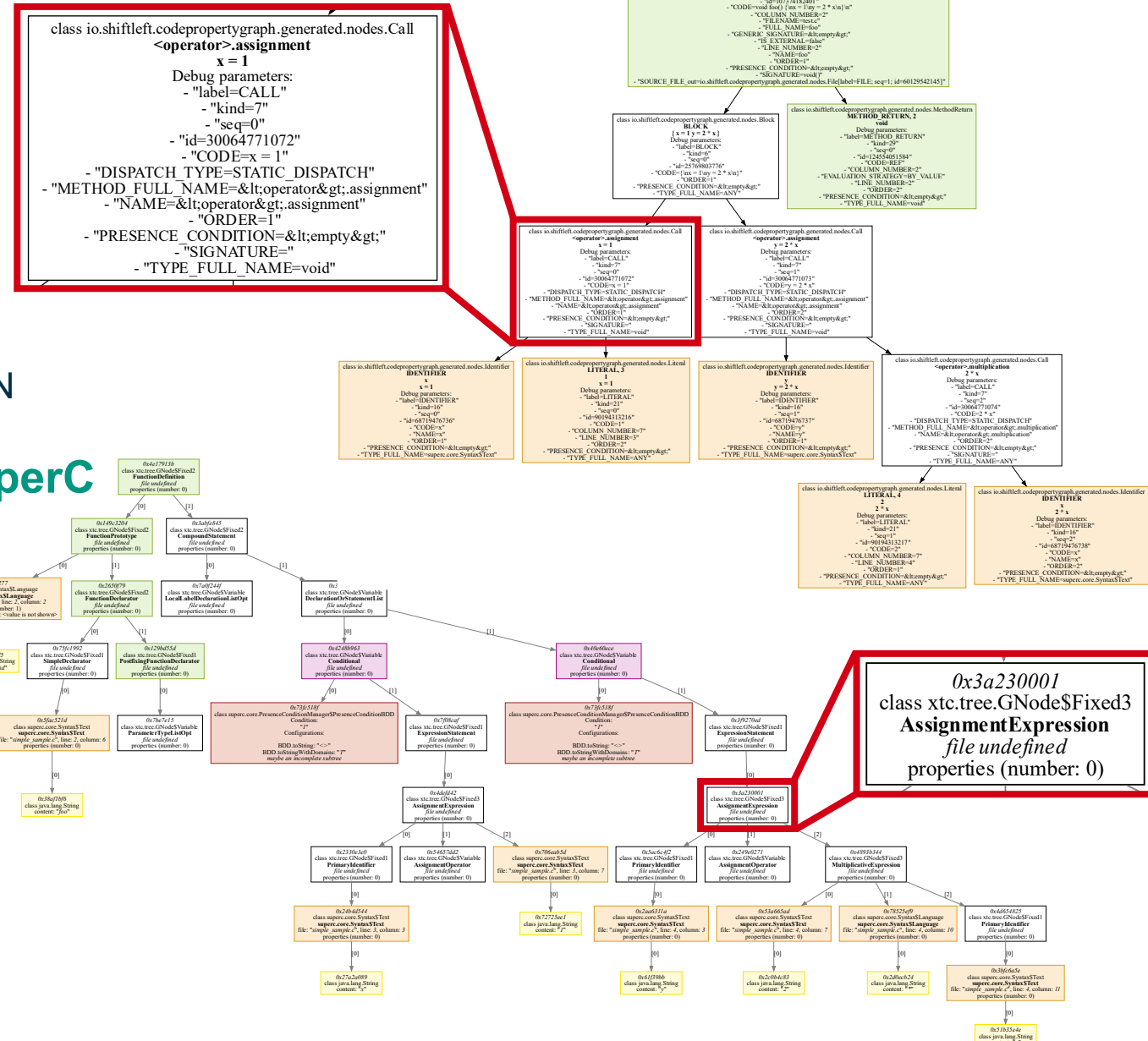
- C features can be treated as **Building blocks** of the programming language
- C features can be **translated feature-wise**
 - In some situations, context is required

3.3 Translation Challenges

AST Granularity Differences

- JOERN creates AST with a functional description
- SuperC creates AST with code syntax resolution
- SuperC has a more detailed AST structure than JOERN

JOERN



Sample Code:

```

1. void foo() {
2.   a = 1;
3.   b = 2 * a;
4. }
  
```

3.3 Translation Challenges

AST Granularity Differences

- JOERN creates AST with a functional description
- SuperC creates AST with code syntax resolution
- SuperC has a more detailed AST structure than JOERN

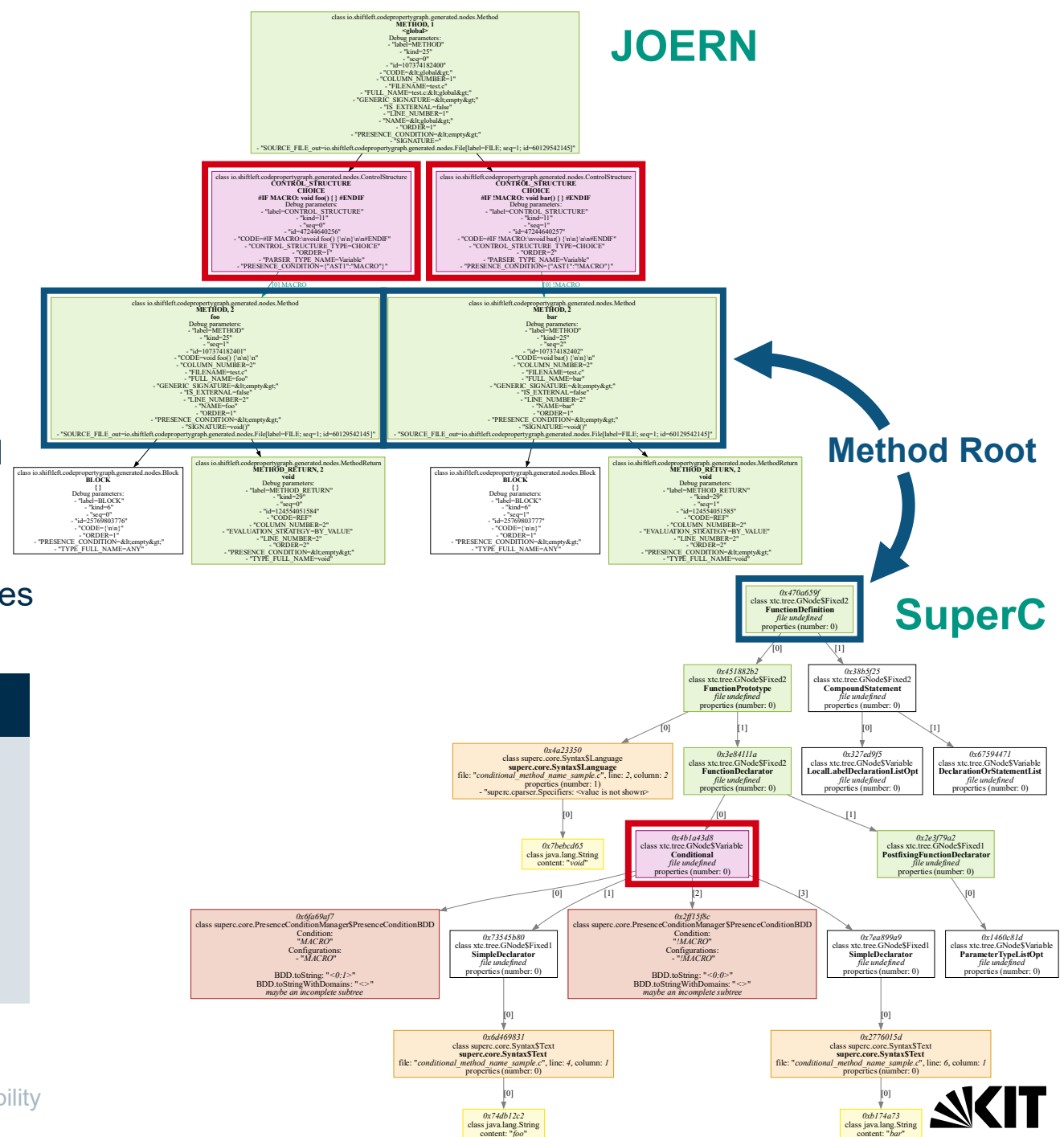
Choice Node Position

- Choice nodes have to be positioned above JOERN nodes

Sample: Variable Function Name

- void
- #if MACRO
- foo
- #else
- bar
- #endif
- () {}

JOERN



SuperC

- JOERN creates AST with a functional description
- SuperC creates AST with code syntax resolution
- SuperC has a more detailed AST structure than JOERN

Choice Node Position

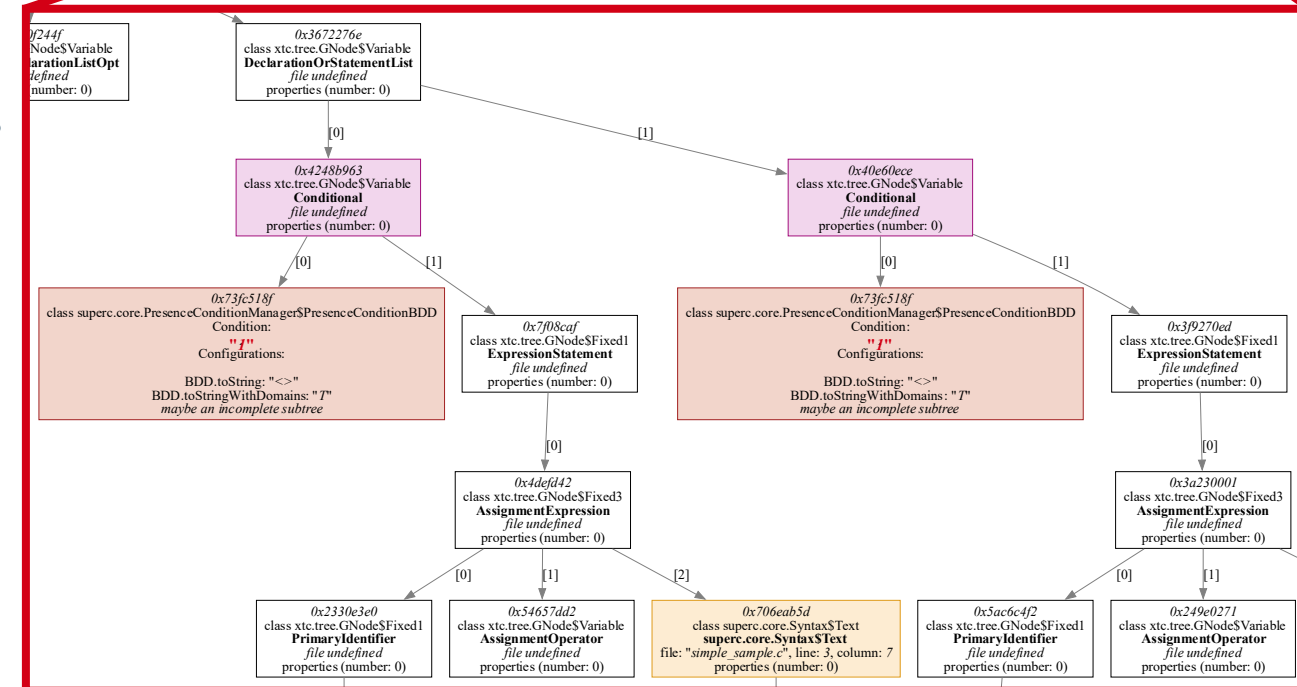
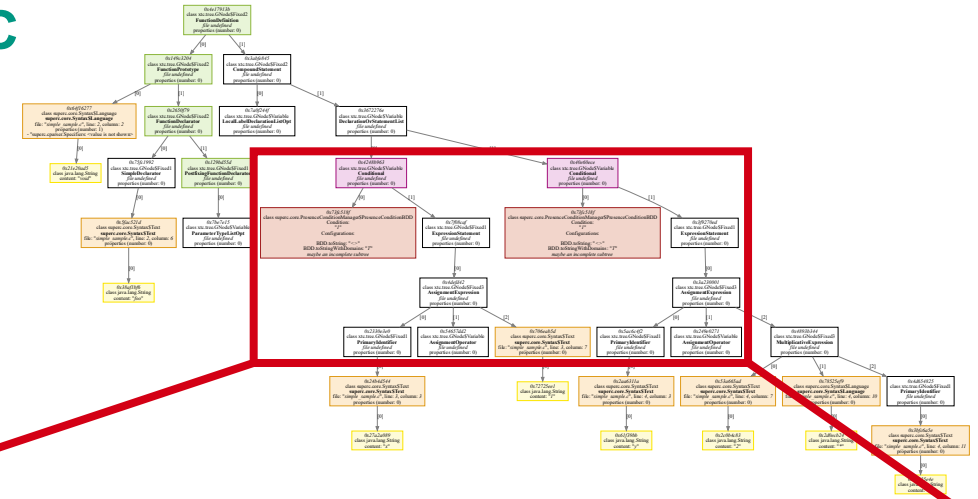
- Choice nodes have to be positioned above JOERN nodes

Ghost Choice Nodes

- SuperC creates, in some situations, unnecessary or duplicated choice nodes

Sample Code:

```
1. void foo() {
2.     a = 1;
3.     b = 2 * a;
4. }
```



3.3 Translation Challenges

AST Granularity Differences

- JOERN creates AST with a functional description
- SuperC creates AST with code syntax resolution
- SuperC has a more detailed AST structure than JOERN

Choice Node Position

- Choice nodes have to be positioned above JOERN nodes

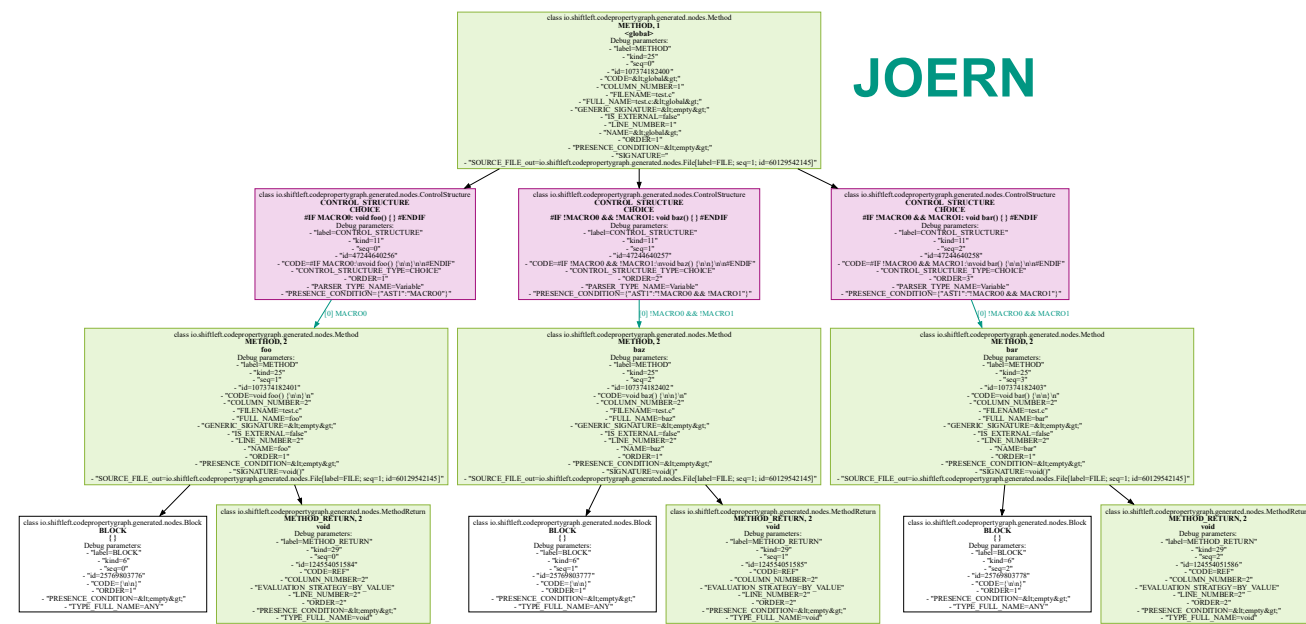
Ghost Choice Nodes

- SuperC creates, in some situations, unnecessary or duplicated choice nodes

Conditional Macro Condition abstraction

- Choice node presence condition abstracts from the code

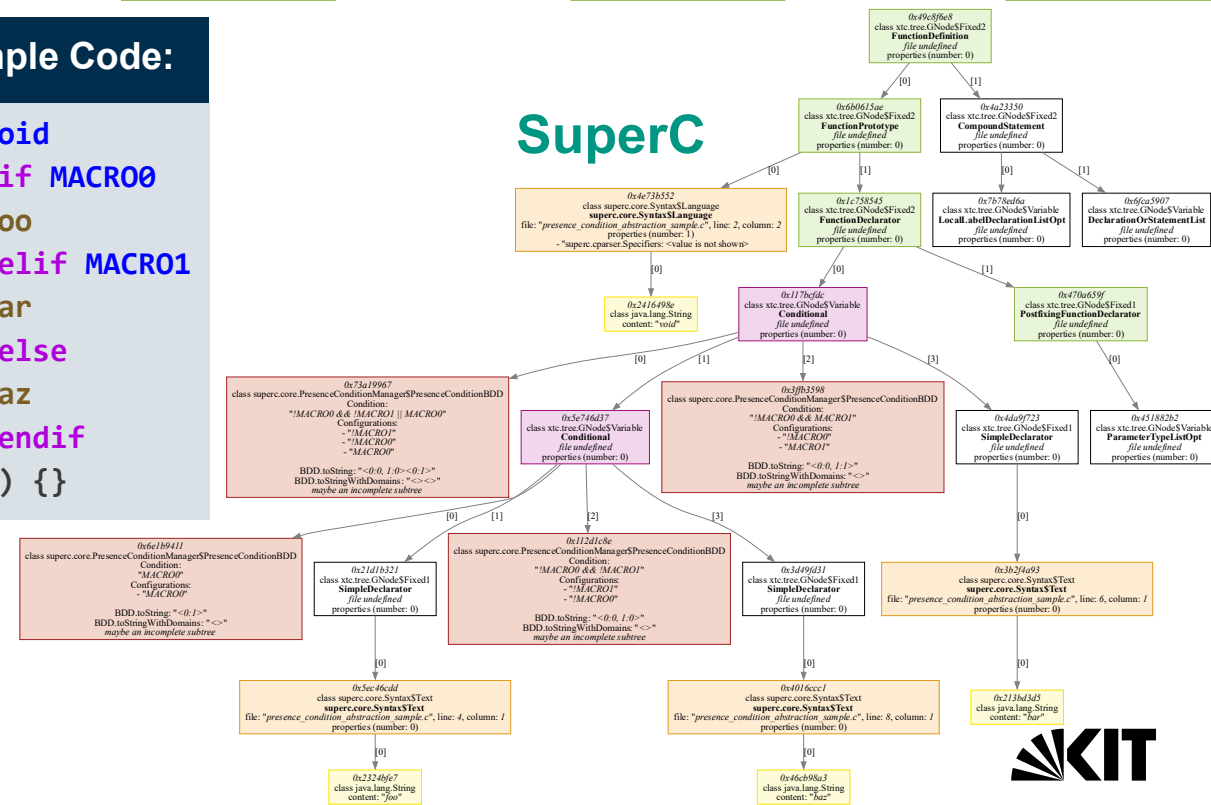
JOERN



Sample Code:

- void
- #if MACRO0
- foo
- #elif MACRO1
- bar
- #else
- baz
- #endif
- () {}

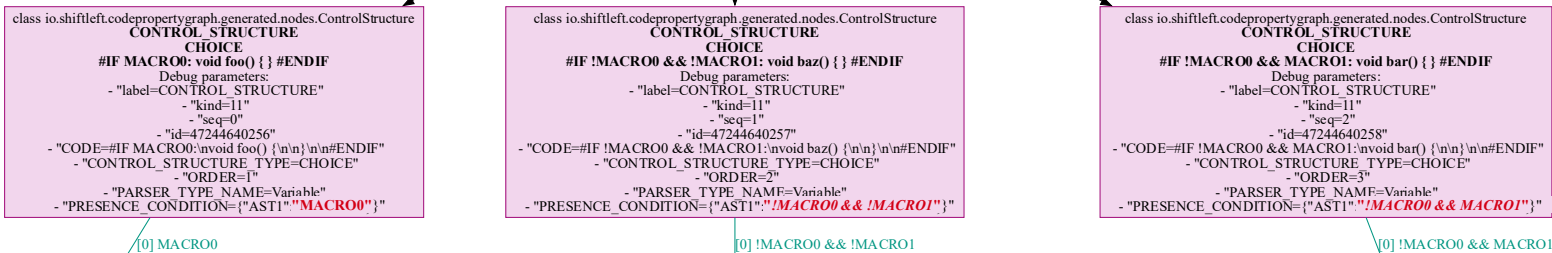
SuperC



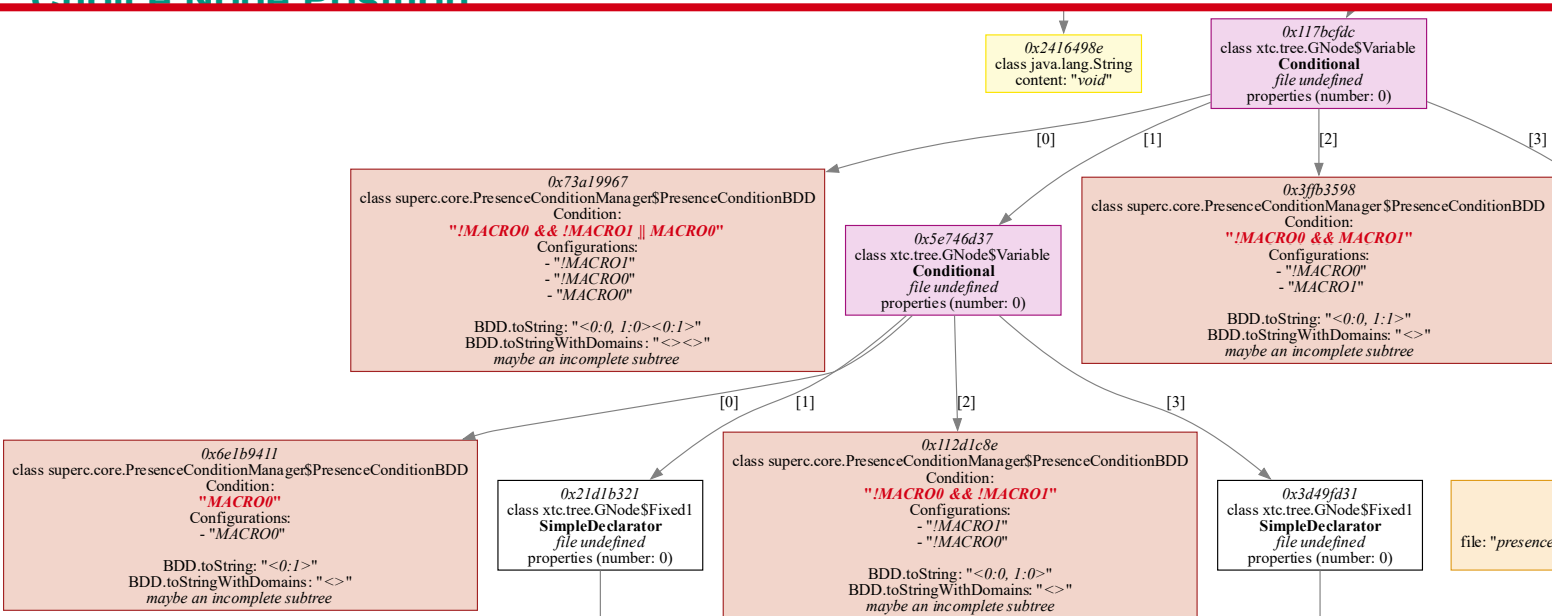
3.3 Translation Challenges

JOERN

AST Granularity Differences



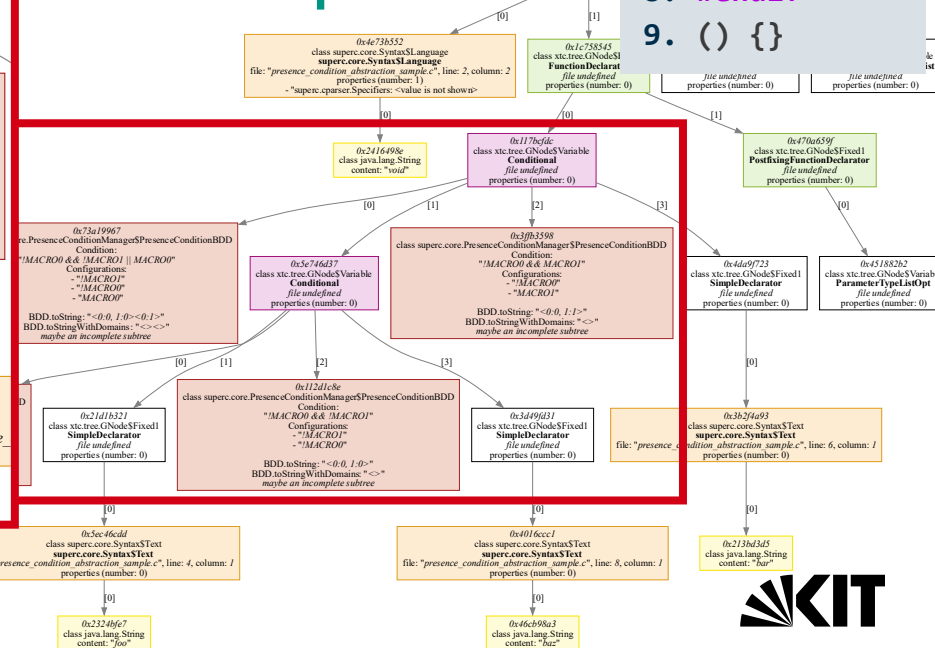
Choice Node Position



Sample Code:

1. void
2. #if MACRO0
3. foo
4. #elif MACRO1
5. bar
6. #else
7. baz
8. #endif
9. () {}

SuperC



3.3 Translation Challenges

AST Granularity Differences

- JOERN creates AST with a functional description
- SuperC creates AST with code syntax resolution
- SuperC has a more detailed AST structure than JOERN

Choice Node Position

- Choice nodes have to be positioned above JOERN nodes

Ghost Choice Nodes

- SuperC creates, in some situations, unnecessary or duplicated choice nodes

Conditional Macro Condition abstraction

- Choice node presence condition abstracts from the code

High variability of C language features

- Examples: method declaration & variable declaration

Sample: Method Variability Options

```
<return type> <pointer>(<method-name>)<array size>(  
    <var-type> <pointer><var-name><array size>,  
    <# parameters>  
) {...}
```

Degrees of Choice Freedom

1. Return type
2. Return pointer degree
3. Return array number of dimensions
4. Return array dimension sizes
5. Method name
6. Number of parameters
7. Parameter type
8. Parameter dereferentiation
9. Parameter array dimensions
10. Parameter array dimension sizes
11. Parameter name

3.3 Translation Challenges

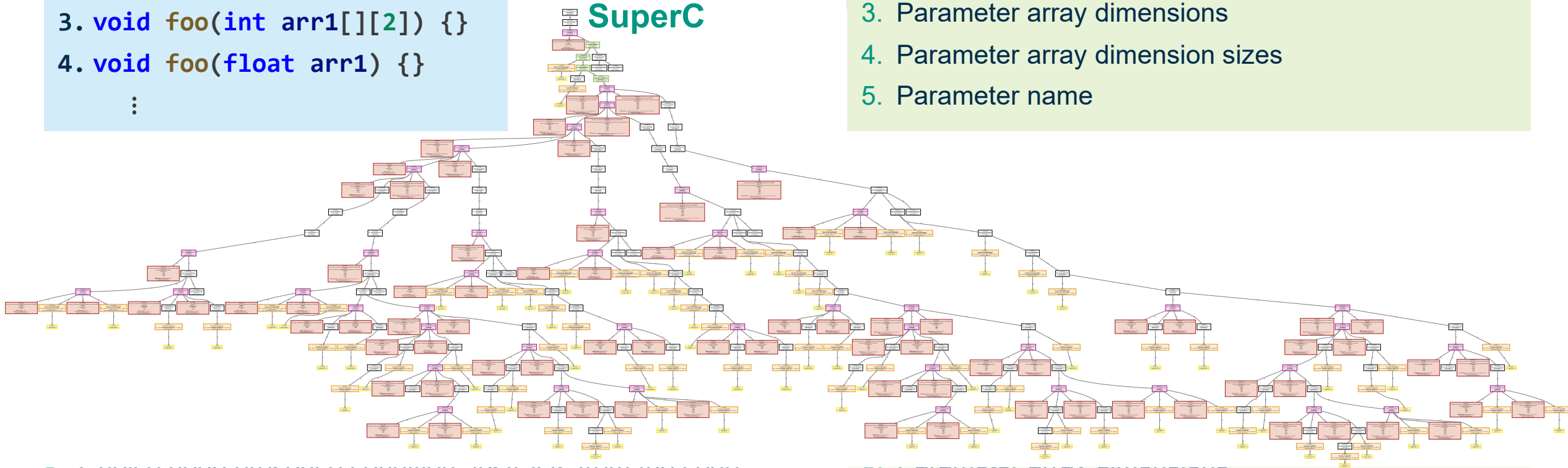
Sample: Method Configurations

1. `void foo(int arr1) {}`
2. `void foo(int ***arr1) {}`
3. `void foo(int arr1[][2]) {}`
4. `void foo(float arr1) {}`
- ⋮

Sample: Method with **ONLY ONE** configurable parameter

1. Parameter type
2. Parameter dereferenciation
3. Parameter array dimensions
4. Parameter array dimension sizes
5. Parameter name

SuperC



- Choice node presence condition abstracts from the code

High variability of C language features

- Examples: method declaration & variable declaration

10. Parameter array dimension sizes
11. Parameter name

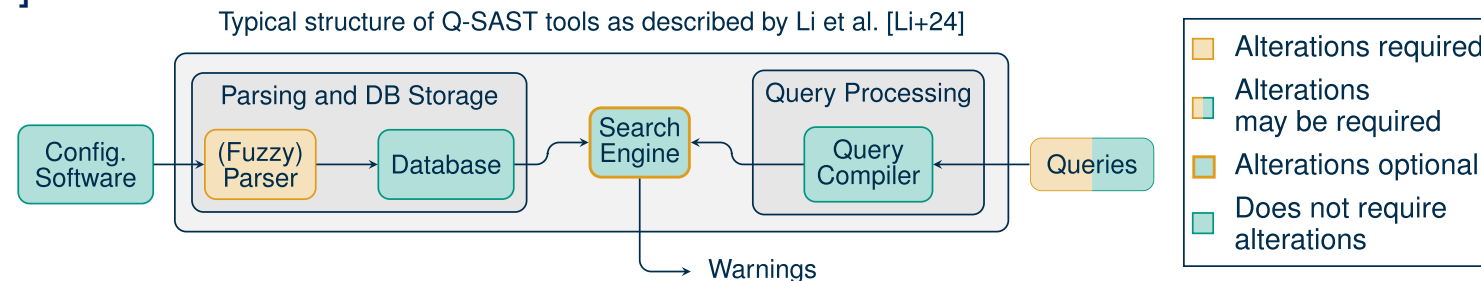
4. Conclusion

Current State

- Full conditional support for **VA-AST creation** except for
 - struct declaration and typedef
 - assembler instructions
 - correct handling of macros (not marked as macro)
 - multi-file handling (is only a small outer AST adaptation)
- Automatic removal and **simplification of presence conditions**
 - The simplification is limited to the logic algebra
- Functionality for SuperC and JOERN **graph visualization**
- Initial modifications of the **VA-CFG, VA-PDG, VA-CPG** from Mats Dormann [Do26]

Future Work

- **Completing** the translation of the missing C features
- **Extending** the existing creation of **VA-CFG, VA-PDG** and **VA-CPG** (extending the handling of conditionals/choice nodes)
 - Function declarations (parameters, return)
 - Function calls (parameters)
 - goto (linking to labels)
 - Control structures (e.g.: if, for, while, ...)
 - variable dependencies
- Adapt the **Query Compiler** and **Search Engine** to support variability code



Literature

- [Li+24] Zongjie Li et al. “Evaluating C/C++ Vulnerability Detectability of Query-Based Static Application Security Testing Tools”. In: IEEE Transactions on Dependable and Secure Computing 21.5 (2024), pp. 1–18. ISSN: 1545-5971, 1941-0018, 2160-9209. DOI: 10.1109/TDSC.2024.3354789. URL: <https://ieeexplore.ieee.org/document/10400834/> (visited on 05/08/2024).
- [Go+14] Reid Goldsborough. Dealing with the Heartbleed Bug. Military Officers Association of America (MOAA), 14 April 2014. URL: <https://www.moaa.org/content/publications-and-media/features-and-columns/tech-tactics/dealing-with-the-heartbleed-bug/>
- [Do26] Mats Oskar Dormann. Family-Based Vulnerability Discovery for Software Product Lines Using Variability-Aware Code Property Graphs. Master’s Thesis, Karlsruhe Institute of Technology (KIT), Department of Informatics, TVA, 2026.
- [Be+24] Aimee Beckwith-Hills. Lessons Learned from the CrowdStrike Global IT Outage. Air IT Group, 26 July 2024. URL: <https://www.airit.co.uk/insights-resources/lessons-learned-from-the-crowdstrike-global-it-outage/>
- [Tz+24] Iason Tzortzis. Log4Shell Vulnerability (CVE-2021-44228) Explained. Medium. URL: <https://medium.com/@iason.tzortzis/log4shell-vulnerability-cve-2021-44228-explained-114bf3c51d87>
- [Bä26] Tim Bächle. “Family-Based Vulnerability Discovery for Highly-Configurable Software Systems”. Kick-Off Slides, Karlsruhe Institute of Technology (KIT), Department of Informatics, TVA, 2026.

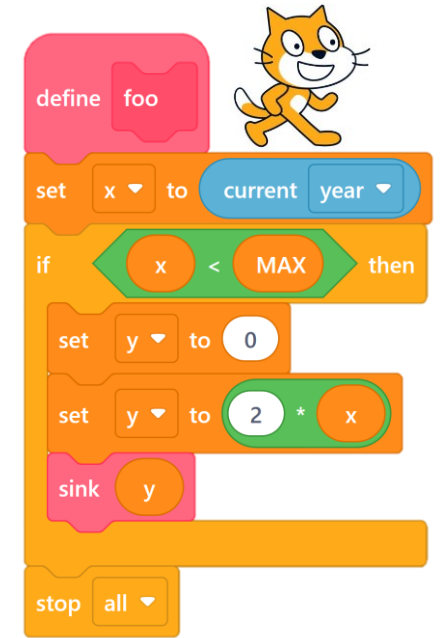


Translation C-Features

- | | | |
|---|--|--|
| <ul style="list-style-type: none">✓ Unary operators
Operators like ++ or !✓ Binary operators
Operators like + or &&✓ Variable declaration✓ Multi variable declaration✗ typedef and struct
typedef and struct declarations and initialization✓ Enum
Enum declarations✗ Simple parameterized macros
definition of macros | <ul style="list-style-type: none">✓ Function call✓ Function declaration✓ Multi function handling✗ Multi file support
Create one VA-AST for multiple C-files✓ Code generation
Code reconstruction from SuperC VA-AST✓ Code block handling
sequence of instruction e.g. if-block✓ Conditional macros
Preprocessor configurations options e.g.:
<code>#if</code>, <code>#elif</code>, <code>#else</code>,
<code>#endif</code>, ... | <ul style="list-style-type: none">✓ if statements✓ switch-case statements✓ while and do-while loop✓ for loop✓ break and continue✓ return statements✓ goto and goto-labels✗ assembler instructions |
|---|--|--|

Tool-Feature

- ✓ **Dot-graph visualization**



Discovery

- C features can be treated as **Building blocks** of the programming language
- C features can be **translated feature-wise**
 - In some situations, context is required

